利用 AI 工具操作内网隔离虚拟机的完整方案，避免内网终端一级联网（禁用虚拟网卡并关闭虚拟机网络适配器后依然可用）

环境：Windows 11 + VMware Workstation Pro（VMtools） + Kali Linux（虚拟机系统）

更新日期：2026-07-22

注意：以下路径均为我本地路径，请根据实际情况修改

目录

1. 需求分析

2. 方案对比：SSH vs VMware Tools

3. 整体架构

4. 环境准备

5. VMware MCP 服务搭建

- 5.1 项目结构

- 5.2 核心源码

- 5.3 启动脚本

- 5.4 MCP Server 配置

- 5.5 验证服务启动

6. 解决核心难题：捕获 Guest 命令输出

7. 日常操作指南

8. 安全边界分析

9. 常见问题 Q&A

10. 总结

---

1. 需求分析

1.1 场景描述



1.2 核心约束

| 约束项 | 说明 |

| --- | --- |

| 虚拟机不允许联网 | 桥接到内网网卡，物理隔离互联网 |

| 宿主机可以联网 | 用于调用云端 LLM API |

| AI 要能操作虚拟机 | 执行命令、查文件、传文件、管理电源状态 |

| 操作不能经过内网网卡 | 内网环境对虚拟机流量零感知 |

| 命令输出必须返回 | AI 需要根据输出来决策下一步 |

1.3 为什么不能用常规方案?

第一直觉是用 SSH：宿主机 SSH 到虚拟机执行命令。但这要求宿主机和虚拟机之间**网络互通**——如果宿主机不在同一个内网段，或者虚拟机的网卡是隔离的，这条路就走不通。

更关键的是：**内网安全的本质需求是虚拟机完全静默，连宿主机到虚拟机的 TCP 流量都不应该出现在内网上。**

---

2. 方案对比：SSH vs VMware Tools

2.1 SSH 方案（已淘汰）

```
宿主机 AI Agent → SSH（paramiko）→ 虚拟机 SSHD → 执行命令 → 返回输出
                  ↓
                  依赖 TCP/IP 网络
                  需要 SSH 服务
                  流量经过网卡
```

| 维度 | 评价 |

| --- | --- |

| 部署复杂度 | 中 — 需配置 SSH、防火墙、网络互通 |

| 安全性 | ❌ 流量经过内网网卡，有网络痕迹 |

| 断网可用 | ❌ 虚拟机断网则无法操作 |

| 电源管理 | ❌ 虚拟机未开机时无法 SSH |

| 文件传输 | ✅ scp/sftp |

| 快照管理 | ❌ 不支持 |

2.2 VMware Tools 方案（本文方案）

```
宿主机 AI Agent → vmrun → VMware VMCI 驱动 → 虚拟机 VMware Tools 服务 → 执行命令
                                                    ↑
                                                不经过网卡
                                                不走 TCP/IP
                                                零网络流量
```

| 维度 | 评价 |

| --- | --- |

| 部署复杂度 | 低 — 只需安装 VMware Tools（默认已装） |

| 安全性 | ✅ **零网络流量，内网完全无感知** |

| 断网可用 | ✅ 拔网线也能执行命令 |

| 电源管理 | ✅ 支持开机、关机、暂停、快照 |

| 文件传输 | ✅ vmrun copy 命令 |

| 快照管理 | ✅ 支持创建、恢复、删除快照 |

| 截图 | ✅ 支持截取虚拟机屏幕 |

2.3 结论

VMware Tools 方案完胜 SSH 方案，尤其是"内网零流量"这个特性，在安全敏感场景中是决定性优势。

---

3. 整体架构

3.1 系统分层



3.2 关键发现

在执行命令获取输出这条线上，踩了一个大坑：

| 尝试的方案 | 结果 |

| --- | --- |

| vmrest REST API Guest Operations | ❌ Workstation Pro 不支持（Fusion/vSphere only） |

| vmcli `Guest run` 子命令 | ❌ 不返回 guest stdout |

| vmrun `runProgramInGuest` | ❌ 不返回 guest stdout（DeepSeek 错误声称可以） |

| vmrun `runScriptInGuest` | ❌ 不返回 guest stdout（同上） |

| **文件式 IPC**（最终方案） | ✅ **成功！重定向到临时文件 → CopyFileFromGuestToHost → 读取 → 清理** |

---

4. 环境准备

4.1 宿主机要求

| 项目 | 要求 |

| --- | --- |

| 操作系统 | Windows 10/11 x64 |

| VMware Workstation | Pro 17.x 或更高 |

| Python | >= 3.10 |

| 虚拟机系统 | 任意（需已安装 VMware Tools） |

| AI 工具 | 任意（需可以正常使用MCP，我用的是opencode） |

4.2 确认 VMware Tools 已安装

在虚拟机中执行：

```
vmware-toolbox-cmd --version
```

如果 VMware Tools 已安装，会返回版本号。如果没有安装，在 VMware Workstation 菜单中：**VM → Install VMware Tools**。

4.3 确认 vmrun 可用

```powershell
# 查找 vmrun 路径
Get-ChildItem -Path "C:\Program Files\VMware" -Recurse -Filter "vmrun.exe" | Select
-ExpandProperty FullName

# 预期输出:
# C:\Program Files\VMware\VMware Workstation\vmrun.exe
```

测试基本功能:

```powershell
& "C:\Program Files\VMware\VMware Workstation\vmrun.exe" list
```

输出应为:

```
Total running VMs: N
VMX 文件路径列表...
```

---

5. VMware MCP 服务搭建

5.1 项目结构

```
D:\opencode\vmware-mcp/
├── pyproject.toml              # 项目配置
├── run-vmware-mcp.bat          # 启动脚本
└── src/
    └── vmware_mcp/
        ├── server.py           # MCP Server 主入口，工具定义与路由
        ├── client.py           # vmrest REST API 客户端
        ├── vmrun.py            # vmrun 命令行包装器（核心！）
        └── vmcli.py            # vmcli 命令行包装器
```

5.2 核心源码

`pyproject.toml` — 项目依赖配置

```toml
[project]
name = "vmware-mcp"
version = "0.1.0"
description = "MCP server for VMware Workstation Pro"
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "mcp>=1.0.0",
    "httpx>=0.27.0",
]

[project.scripts]
vmware-mcp = "vmware_mcp.server:main"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

`client.py` — vmrest REST API 客户端

```python
import httpx
from typing import Any


class VMwareClient:
    """HTTP client for VMware Workstation Pro REST API."""

    def __init__(self, host: str = "localhost", port: int = 8697,
                 username: str = "", password: str = ""):
        self.base_url = f"http://{host}:{port}/api"
        self.auth = (username, password) if username else None

    async def _request(self, method: str, path: str, **kwargs) -> Any:
        async with httpx.AsyncClient(auth=self.auth, verify=False) as client:
            resp = await client.request(method, f"{self.base_url}{path}", **kwargs)
            resp.raise_for_status()
            if resp.content:
                return resp.json()
            return None

    # VM Management
    async def list_vms(self) -> list[dict]:
        return await self._request("GET", "/vms")

    async def get_vm(self, vm_id: str) -> dict:
        return await self._request("GET", f"/vms/{vm_id}")

    async def create_vm(self, vm_id: str, name: str) -> dict:
        return await self._request("POST", f"/vms/{vm_id}", json={"name": name})

    async def delete_vm(self, vm_id: str) -> None:
        await self._request("DELETE", f"/vms/{vm_id}")

    async def update_vm(self, vm_id: str, settings: dict) -> dict:
        return await self._request("PUT", f"/vms/{vm_id}", json=settings)

    # VM Power
    async def get_power_state(self, vm_id: str) -> dict:
        return await self._request("GET", f"/vms/{vm_id}/power")

    async def change_power_state(self, vm_id: str, state: str) -> dict:
        return await self._request("PUT", f"/vms/{vm_id}/power", params={"state":
state})

    # VM Network Adapters
    async def list_nics(self, vm_id: str) -> list[dict]:
        return await self._request("GET", f"/vms/{vm_id}/nic")

    async def create_nic(self, vm_id: str, nic_config: dict) -> dict:
        return await self._request("POST", f"/vms/{vm_id}/nic", json=nic_config)

    async def update_nic(self, vm_id: str, index: int, nic_config: dict) -> dict:
        return await self._request("PUT", f"/vms/{vm_id}/nic/{index}",
```

```
json=nic_config)

    async def delete_nic(self, vm_id: str, index: int) -> None:
        await self._request("DELETE", f"/vms/{vm_id}/nic/{index}")

    async def get_vm_ip(self, vm_id: str) -> dict:
        return await self._request("GET", f"/vms/{vm_id}/ip")

    # VM Shared Folders
    async def list_shared_folders(self, vm_id: str) -> list[dict]:
        return await self._request("GET", f"/vms/{vm_id}/sharedfolders")

    async def create_shared_folder(self, vm_id: str, folder_config: dict) -> dict:
        return await self._request("POST", f"/vms/{vm_id}/sharedfolders",
json=folder_config)

    async def update_shared_folder(self, vm_id: str, folder_id: str, folder_config:
dict) -> dict:
        return await self._request("PUT",
f"/vms/{vm_id}/sharedfolders/{folder_id}", json=folder_config)

    async def delete_shared_folder(self, vm_id: str, folder_id: str) -> None:
        await self._request("DELETE", f"/vms/{vm_id}/sharedfolders/{folder_id}")

    # Host Networks
    async def list_networks(self) -> list[dict]:
        return await self._request("GET", "/vmnet")

    async def create_network(self, network_config: dict) -> dict:
        return await self._request("POST", "/vmnets", json=network_config)

    async def get_mac_to_ips(self, vmnet: str) -> list[dict]:
        return await self._request("GET", f"/vmnet/{vmnet}/mactoip")

    async def update_mac_to_ip(self, vmnet: str, mac: str, ip: str) -> dict:
        return await self._request("PUT", f"/vmnet/{vmnet}/mactoip/{mac}", json=
{"ip": ip})

    async def get_portforwards(self, vmnet: str) -> list[dict]:
        return await self._request("GET", f"/vmnet/{vmnet}/portforward")

    async def update_portforward(self, vmnet: str, protocol: str, port: int,
config: dict) -> dict:
        return await self._request("PUT",
f"/vmnet/{vmnet}/portforward/{protocol}/{port}", json=config)

    async def delete_portforward(self, vmnet: str, protocol: str, port: int) ->
None:
        await self._request("DELETE",
f"/vmnet/{vmnet}/portforward/{protocol}/{port}")
```

`vmrun.py` — vmrun 命令行包装器（核心文件）

```python
import asyncio
import os
import tempfile


class VMRun:
    """Wrapper for vmrun command line tool."""

    def __init__(self, vmrun_path: str | None = None):
        self.vmrun_path = vmrun_path or os.getenv(
            "VMRUN_PATH",
            r"C:\Program Files\VMware\VMware Workstation\vmrun.exe"
        )

    async def _run(self, command: str, *args: str,
                   guest_user: str = "", guest_pass: str = "") -> str:
        cmd = [self.vmrun_path, "-T", "ws"]
        if guest_user:
            cmd.extend(["-gu", guest_user])
        if guest_pass:
            cmd.extend(["-gp", guest_pass])
        cmd.append(command)
        cmd.extend(args)

        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await proc.communicate()

        if proc.returncode != 0:
            error_msg = stderr.decode("utf-8", errors="replace").strip()
            if not error_msg:
                error_msg = stdout.decode("utf-8", errors="replace").strip()
            raise RuntimeError(f"vmrun failed: {error_msg}")

        return stdout.decode("utf-8", errors="replace").strip()

    # === Power ===
    async def start(self, vmx_path: str, gui: bool = True) -> str:
        return await self._run("start", vmx_path, "gui" if gui else "nogui")

    async def stop(self, vmx_path: str, hard: bool = False) -> str:
        return await self._run("stop", vmx_path, "hard" if hard else "soft")

    async def reset(self, vmx_path: str, hard: bool = False) -> str:
        return await self._run("reset", vmx_path, "hard" if hard else "soft")

    async def suspend(self, vmx_path: str, hard: bool = False) -> str:
        return await self._run("suspend", vmx_path, "hard" if hard else "soft")

    async def pause(self, vmx_path: str) -> str:
```

```
        return await self._run("pause", vmx_path)

    async def unpause(self, vmx_path: str) -> str:
        return await self._run("unpause", vmx_path)

    # === General ===
    async def list_running(self) -> str:
        return await self._run("list")

    async def upgrade_vm(self, vmx_path: str) -> str:
        return await self._run("upgradevm", vmx_path)

    async def delete_vm(self, vmx_path: str) -> str:
        return await self._run("deleteVM", vmx_path)

    async def clone(self, vmx_path: str, dest_path: str,
                    clone_type: str = "linked", snapshot: str = "",
                    clone_name: str = "") -> str:
        args = [vmx_path, dest_path, clone_type]
        if snapshot:
            args.append(f"-snapshot={snapshot}")
        if clone_name:
            args.append(f"-cloneName={clone_name}")
        return await self._run("clone", *args)

    # === Snapshot ===
    async def list_snapshots(self, vmx_path: str, show_tree: bool = False) -> str:
        args = [vmx_path]
        if show_tree:
            args.append("showTree")
        return await self._run("listSnapshots", *args)

    async def snapshot(self, vmx_path: str, name: str) -> str:
        return await self._run("snapshot", vmx_path, name)

    async def delete_snapshot(self, vmx_path: str, name: str,
                              delete_children: bool = False) -> str:
        args = [vmx_path, name]
        if delete_children:
            args.append("andDeleteChildren")
        return await self._run("deleteSnapshot", *args)

    async def revert_to_snapshot(self, vmx_path: str, name: str) -> str:
        return await self._run("revertToSnapshot", vmx_path, name)

    # === Guest File Operations ===
    async def file_exists(self, vmx_path: str, guest_path: str,
                          user: str = "", password: str = "") -> str:
        return await self._run("fileExistsInGuest", vmx_path, guest_path,
                               guest_user=user, guest_pass=password)

    async def directory_exists(self, vmx_path: str, guest_path: str,
                               user: str = "", password: str = "") -> str:
        return await self._run("directoryExistsInGuest", vmx_path, guest_path,
                               guest_user=user, guest_pass=password)
```

```
    async def rename_file(self, vmx_path: str, old_path: str, new_path: str,
                          user: str = "", password: str = "") -> str:
        return await self._run("renameFileInGuest", vmx_path, old_path, new_path,
                               guest_user=user, guest_pass=password)

    async def create_temp_file(self, vmx_path: str,
                               user: str = "", password: str = "") -> str:
        return await self._run("CreateTempfileInGuest", vmx_path,
                               guest_user=user, guest_pass=password)

    async def list_directory(self, vmx_path: str, guest_path: str,
                             user: str = "", password: str = "") -> str:
        return await self._run("listDirectoryInGuest", vmx_path, guest_path,
                               guest_user=user, guest_pass=password)

    async def create_directory(self, vmx_path: str, guest_path: str,
                               user: str = "", password: str = "") -> str:
        return await self._run("createDirectoryInGuest", vmx_path, guest_path,
                               guest_user=user, guest_pass=password)

    async def delete_directory(self, vmx_path: str, guest_path: str,
                               user: str = "", password: str = "") -> str:
        return await self._run("deleteDirectoryInGuest", vmx_path, guest_path,
                               guest_user=user, guest_pass=password)

    async def delete_file(self, vmx_path: str, guest_path: str,
                          user: str = "", password: str = "") -> str:
        return await self._run("deleteFileInGuest", vmx_path, guest_path,
                               guest_user=user, guest_pass=password)

    async def copy_to_guest(self, vmx_path: str, host_path: str, guest_path: str,
                            user: str = "", password: str = "") -> str:
        return await self._run("CopyFileFromHostToGuest", vmx_path, host_path,
guest_path,
                               guest_user=user, guest_pass=password)

    async def copy_from_guest(self, vmx_path: str, guest_path: str, host_path: str,
                              user: str = "", password: str = "") -> str:
        return await self._run("CopyFileFromGuestToHost", vmx_path, guest_path,
host_path,
                               guest_user=user, guest_pass=password)

    # === File-based IPC for capturing guest stdout ===
    async def _run_with_output(self, vmx_path: str, shell_cmd: str,
                               user: str = "", password: str = "") -> str:
        await self._run("runScriptInGuest", vmx_path, "/bin/sh",
                        f"{shell_cmd} > /tmp/_vout 2>&1",
                        guest_user=user, guest_pass=password)

        local_tmp = os.path.join(tempfile.gettempdir(),
f"_vmrun_vout_{os.getpid()}")
        output = ""
        try:
            try:
                await self._run("CopyFileFromGuestToHost", vmx_path,
                                "/tmp/_vout", local_tmp,
```

```
                                guest_user=user, guest_pass=password)
            except RuntimeError:
                pass
            if os.path.exists(local_tmp):
                with open(local_tmp, "r", encoding="utf-8", errors="replace") as f:
                    output = f.read().strip()
        finally:
            if os.path.exists(local_tmp):
                os.remove(local_tmp)
            await self._run("runScriptInGuest", vmx_path, "/bin/sh",
                            "rm -f /tmp/_vout",
                            guest_user=user, guest_pass=password)
        return output

    # === Guest Process Operations ===
    async def run_program(self, vmx_path: str, program: str, args: str = "",
                          no_wait: bool = False, active_window: bool = False,
                          interactive: bool = False,
                          user: str = "", password: str = "") -> str:
        if no_wait:
            cmd_args = [vmx_path, "-noWait"]
            if active_window:
                cmd_args.append("-activeWindow")
            if interactive:
                cmd_args.append("-interactive")
            cmd_args.append(program)
            if args:
                cmd_args.extend(args.split())
            return await self._run("runProgramInGuest", *cmd_args,
                                   guest_user=user, guest_pass=password)

        shell_cmd = f"exec {program}"
        if args:
            shell_cmd += f" {args}"
        return await self._run_with_output(vmx_path, shell_cmd, user, password)

    async def run_script(self, vmx_path: str, interpreter: str, script: str,
                         no_wait: bool = False, active_window: bool = False,
                         interactive: bool = False,
                         user: str = "", password: str = "") -> str:
        if no_wait:
            cmd_args = [vmx_path, "-noWait"]
            if active_window:
                cmd_args.append("-activeWindow")
            if interactive:
                cmd_args.append("-interactive")
            cmd_args.extend([interpreter, script])
            return await self._run("runScriptInGuest", *cmd_args,
                                   guest_user=user, guest_pass=password)

        return await self._run_with_output(vmx_path, script, user, password)

    async def list_processes(self, vmx_path: str,
                             user: str = "", password: str = "") -> str:
        return await self._run("listProcessesInGuest", vmx_path,
                               guest_user=user, guest_pass=password)
```

```
    async def kill_process(self, vmx_path: str, pid: int,
                           user: str = "", password: str = "") -> str:
        return await self._run("killProcessInGuest", vmx_path, str(pid),
                               guest_user=user, guest_pass=password)

    # === Shared Folders ===
    async def enable_shared_folders(self, vmx_path: str) -> str:
        return await self._run("enableSharedFolders", vmx_path)

    async def disable_shared_folders(self, vmx_path: str) -> str:
        return await self._run("disableSharedFolders", vmx_path)

    async def add_shared_folder(self, vmx_path: str, name: str, host_path: str) ->
str:
        return await self._run("addSharedFolder", vmx_path, name, host_path)

    async def remove_shared_folder(self, vmx_path: str, name: str) -> str:
        return await self._run("removeSharedFolder", vmx_path, name)

    async def set_shared_folder_state(self, vmx_path: str, name: str,
                                      host_path: str, writable: bool = True) ->
str:
        return await self._run("setSharedFolderState", vmx_path, name, host_path,
                               "writable" if writable else "readonly")

    # === Device ===
    async def connect_device(self, vmx_path: str, device_name: str) -> str:
        return await self._run("connectNamedDevice", vmx_path, device_name)

    async def disconnect_device(self, vmx_path: str, device_name: str) -> str:
        return await self._run("disconnectNamedDevice", vmx_path, device_name)

    # === Variables ===
    async def read_variable(self, vmx_path: str, var_type: str, name: str,
                            user: str = "", password: str = "") -> str:
        return await self._run("readVariable", vmx_path, var_type, name,
                               guest_user=user, guest_pass=password)

    async def write_variable(self, vmx_path: str, var_type: str, name: str, value:
str,
                             user: str = "", password: str = "") -> str:
        return await self._run("writeVariable", vmx_path, var_type, name, value,
                               guest_user=user, guest_pass=password)

    # === Screen/Input ===
    async def capture_screen(self, vmx_path: str, output_path: str) -> str:
        return await self._run("captureScreen", vmx_path, output_path)

    async def type_keystrokes(self, vmx_path: str, keystrokes: str) -> str:
        return await self._run("typeKeystrokesInGuest", vmx_path, keystrokes)

    # === Tools ===
    async def install_tools(self, vmx_path: str) -> str:
        return await self._run("installTools", vmx_path)
```

```
    async def check_tools_state(self, vmx_path: str) -> str:
        return await self._run("checkToolsState", vmx_path)

    # === Network ===
    async def get_guest_ip(self, vmx_path: str, wait: bool = False) -> str:
        args = [vmx_path]
        if wait:
            args.append("-wait")
        return await self._run("getGuestIPAddress", *args)

    async def list_host_networks(self) -> str:
        return await self._run("listHostNetworks")

    async def list_port_forwardings(self, network: str) -> str:
        return await self._run("listPortForwardings", network)

    async def set_port_forwarding(self, network: str, protocol: str, host_port:
int,
                                  guest_ip: str, guest_port: int,
                                  description: str = "") -> str:
        args = [network, protocol, str(host_port), guest_ip, str(guest_port)]
        if description:
            args.append(description)
        return await self._run("setPortForwarding", *args)

    async def delete_port_forwarding(self, network: str, protocol: str,
                                     host_port: int) -> str:
        return await self._run("deletePortForwarding", network, protocol,
str(host_port))
```

`vmcli.py` — vmcli 命令行包装器

```python
import asyncio
import os
import json
from typing import Any


class VMCli:
    """Wrapper for vmcli command line tool."""

    def __init__(self, vmcli_path: str | None = None):
        self.vmcli_path = vmcli_path or os.getenv(
            "VMCLI_PATH",
            r"C:\Program Files\VMware\VMware Workstation\vmcli.exe"
        )

    async def _run(self, vmx_path: str | None, module: str,
                   command: str, *args: str) -> str:
        cmd = [self.vmcli_path]
        if vmx_path:
            cmd.append(vmx_path)
        cmd.extend([module, command])
        cmd.extend(args)

        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await proc.communicate()

        if proc.returncode != 0:
            error_msg = stderr.decode("utf-8", errors="replace").strip()
            raise RuntimeError(f"vmcli failed: {error_msg}")

        return stdout.decode("utf-8", errors="replace").strip()

    # === Snapshot ===
    async def snapshot_list(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Snapshot", "query")

    async def snapshot_take(self, vmx_path: str, name: str) -> str:
        return await self._run(vmx_path, "Snapshot", "Take", "-n", name)

    async def snapshot_revert(self, vmx_path: str, name: str) -> str:
        return await self._run(vmx_path, "Snapshot", "Revert", "-n", name)

    async def snapshot_delete(self, vmx_path: str, name: str,
                              delete_children: bool = False) -> str:
        args = ["-n", name]
        if delete_children:
            args.append("--andDeleteChildren")
        return await self._run(vmx_path, "Snapshot", "Delete", *args)
```

```
    async def snapshot_clone(self, vmx_path: str, snapshot_name: str,
                             dest_path: str, clone_type: str = "linked") -> str:
        return await self._run(vmx_path, "Snapshot", "Clone",
                               "-n", snapshot_name, "-p", dest_path, "-t",
clone_type)

    # === Guest Operations ===
    async def guest_run(self, vmx_path: str, program: str, args: str = "",
                        user: str = "", password: str = "") -> str:
        cmd_args = ["-p", program]
        if args:
            cmd_args.extend(["-a", args])
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "run", *cmd_args)

    async def guest_ps(self, vmx_path: str,
                       user: str = "", password: str = "") -> str:
        cmd_args = []
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "ps", *cmd_args)

    async def guest_kill(self, vmx_path: str, pid: int,
                         user: str = "", password: str = "") -> str:
        cmd_args = ["--pid", str(pid)]
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "kill", *cmd_args)

    async def guest_ls(self, vmx_path: str, path: str,
                       user: str = "", password: str = "") -> str:
        cmd_args = ["-d", path]
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "ls", *cmd_args)

    async def guest_mkdir(self, vmx_path: str, path: str,
                          user: str = "", password: str = "") -> str:
        cmd_args = ["-d", path]
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "mkdir", *cmd_args)

    async def guest_rm(self, vmx_path: str, path: str,
                       user: str = "", password: str = "") -> str:
```

```
        cmd_args = ["-f", path]
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "rm", *cmd_args)

    async def guest_rmdir(self, vmx_path: str, path: str,
                          user: str = "", password: str = "") -> str:
        cmd_args = ["-d", path]
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "rmdir", *cmd_args)

    async def guest_copy_to(self, vmx_path: str, host_path: str, guest_path: str,
                            user: str = "", password: str = "") -> str:
        cmd_args = ["-l", host_path, "-r", guest_path]
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "copyTo", *cmd_args)

    async def guest_copy_from(self, vmx_path: str, guest_path: str, host_path: str,
                              user: str = "", password: str = "") -> str:
        cmd_args = ["-r", guest_path, "-l", host_path]
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "copyFrom", *cmd_args)

    async def guest_env(self, vmx_path: str,
                        user: str = "", password: str = "") -> str:
        cmd_args = []
        if user:
            cmd_args.extend(["-u", user])
        if password:
            cmd_args.extend(["-P", password])
        return await self._run(vmx_path, "Guest", "env", *cmd_args)

    # === MKS (Mouse, Keyboard, Screen) ===
    async def mks_screenshot(self, vmx_path: str, output_path: str) -> str:
        return await self._run(vmx_path, "MKS", "captureScreenshot", "-o",
output_path)

    async def mks_send_key(self, vmx_path: str, key_sequence: str) -> str:
        return await self._run(vmx_path, "MKS", "sendKeySequence", "-s",
key_sequence)

    async def mks_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "MKS", "query")

    # === Chipset (CPU/Memory) ===
```

```
    async def chipset_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Chipset", "query")

    async def chipset_set_cpu(self, vmx_path: str, count: int) -> str:
        return await self._run(vmx_path, "Chipset", "SetVCpuCount", "-c",
str(count))

    async def chipset_set_memory(self, vmx_path: str, size_mb: int) -> str:
        return await self._run(vmx_path, "Chipset", "SetMemSize", "-s",
str(size_mb))

    async def chipset_set_cores_per_socket(self, vmx_path: str, cores: int) -> str:
        return await self._run(vmx_path, "Chipset", "SetCoresPerSocket", "-c",
str(cores))

    # === Tools ===
    async def tools_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Tools", "Query")

    async def tools_install(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Tools", "Install")

    async def tools_upgrade(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Tools", "Upgrade")

    # === VMTemplate ===
    async def template_create(self, vmx_path: str, template_path: str, name: str) -
> str:
        return await self._run(vmx_path, "VMTemplate", "Create", "-p",
template_path, "-n", name)

    async def template_deploy(self, template_path: str, dest_path: str, name: str)
-> str:
        return await self._run(None, "VMTemplate", "Deploy", "-p", template_path,
"-d", dest_path, "-n", name)

    # === Disk ===
    async def disk_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Disk", "query")

    async def disk_create(self, vmx_path: str, size_gb: int, disk_type: str =
"scsi",
                          adapter: int = 0, device: int = 0) -> str:
        return await self._run(vmx_path, "Disk", "Create", "-s", str(size_gb),
                               "-t", disk_type, "-a", str(adapter), "-d",
str(device))

    async def disk_extend(self, vmx_path: str, new_size_gb: int,
                          adapter: int = 0, device: int = 0) -> str:
        return await self._run(vmx_path, "Disk", "Extend", "-s", str(new_size_gb),
                               "-a", str(adapter), "-d", str(device))

    # === ConfigParams ===
    async def config_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "ConfigParams", "query")
```

```
    async def config_set(self, vmx_path: str, key: str, value: str) -> str:
        return await self._run(vmx_path, "ConfigParams", "SetEntry", "-k", key, "-
v", value)

    # === Power ===
    async def power_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Power", "query")

    async def power_start(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Power", "Start")

    async def power_stop(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Power", "Stop")

    async def power_pause(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Power", "Pause")

    async def power_unpause(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Power", "Unpause")

    async def power_reset(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Power", "Reset")

    async def power_suspend(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Power", "Suspend")

    # === Ethernet ===
    async def ethernet_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Ethernet", "query")

    async def ethernet_set_connection_type(self, vmx_path: str, index: int,
                                           conn_type: str) -> str:
        return await self._run(vmx_path, "Ethernet", "SetConnectionType",
                               "-i", str(index), "-t", conn_type)

    async def ethernet_set_present(self, vmx_path: str, index: int,
                                   present: bool) -> str:
        return await self._run(vmx_path, "Ethernet", "SetPresent",
                               "-i", str(index), "-e", "true" if present else
"false")

    async def ethernet_set_start_connected(self, vmx_path: str, index: int,
                                           connected: bool) -> str:
        return await self._run(vmx_path, "Ethernet", "SetStartConnected",
                               "-i", str(index), "-e", "true" if connected else
"false")

    async def ethernet_set_virtual_device(self, vmx_path: str, index: int,
                                          device: str) -> str:
        return await self._run(vmx_path, "Ethernet", "SetVirtualDevice",
                               "-i", str(index), "-d", device)

    async def ethernet_set_network_name(self, vmx_path: str, index: int,
                                        name: str) -> str:
        return await self._run(vmx_path, "Ethernet", "SetNetworkName",
                               "-i", str(index), "-n", name)
```

```
    async def ethernet_purge(self, vmx_path: str, index: int) -> str:
        return await self._run(vmx_path, "Ethernet", "Purge", "-i", str(index))

    # === HGFS (Shared Folders) ===
    async def hgfs_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "HGFS", "query")

    async def hgfs_set_enabled(self, vmx_path: str, index: int,
                               enabled: bool) -> str:
        return await self._run(vmx_path, "HGFS", "SetEnabled",
                               "-i", str(index), "-e", "true" if enabled else
"false")

    async def hgfs_set_host_path(self, vmx_path: str, index: int, path: str) ->
str:
        return await self._run(vmx_path, "HGFS", "SetHostPath",
                               "-i", str(index), "-p", path)

    async def hgfs_set_guest_name(self, vmx_path: str, index: int, name: str) ->
str:
        return await self._run(vmx_path, "HGFS", "SetGuestName",
                               "-i", str(index), "-n", name)

    async def hgfs_set_read_access(self, vmx_path: str, index: int,
                                   read: bool) -> str:
        return await self._run(vmx_path, "HGFS", "SetReadAccess",
                               "-i", str(index), "-e", "true" if read else "false")

    async def hgfs_set_write_access(self, vmx_path: str, index: int,
                                    write: bool) -> str:
        return await self._run(vmx_path, "HGFS", "SetWriteAccess",
                               "-i", str(index), "-e", "true" if write else
"false")

    # === Serial ===
    async def serial_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Serial", "Query")

    async def serial_set_present(self, vmx_path: str, index: int,
                                 present: bool) -> str:
        return await self._run(vmx_path, "Serial", "SetPresent",
                               "-i", str(index), "-e", "true" if present else
"false")

    async def serial_purge(self, vmx_path: str, index: int) -> str:
        return await self._run(vmx_path, "Serial", "Purge", "-i", str(index))

    # === Sata ===
    async def sata_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Sata", "query")

    async def sata_set_present(self, vmx_path: str, adapter: int,
                               present: bool) -> str:
        return await self._run(vmx_path, "Sata", "SetPresent",
                               "-a", str(adapter), "-e", "true" if present else
```

```
"false")

    async def sata_purge(self, vmx_path: str, adapter: int) -> str:
        return await self._run(vmx_path, "Sata", "Purge", "-a", str(adapter))

    # === Nvme ===
    async def nvme_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "Nvme", "query")

    async def nvme_set_present(self, vmx_path: str, adapter: int,
                               present: bool) -> str:
        return await self._run(vmx_path, "Nvme", "SetPresent",
                               "-a", str(adapter), "-e", "true" if present else
"false")

    async def nvme_purge(self, vmx_path: str, adapter: int) -> str:
        return await self._run(vmx_path, "Nvme", "Purge", "-a", str(adapter))

    # === VProbes ===
    async def vprobes_query(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "VProbes", "Query")

    async def vprobes_set_enabled(self, vmx_path: str, enabled: bool) -> str:
        return await self._run(vmx_path, "VProbes", "SetEnabled",
                               "-e", "true" if enabled else "false")

    async def vprobes_load(self, vmx_path: str, script_path: str) -> str:
        return await self._run(vmx_path, "VProbes", "Load", "-s", script_path)

    async def vprobes_reset(self, vmx_path: str) -> str:
        return await self._run(vmx_path, "VProbes", "Reset")
```

`server.py` — MCP Server 入口（工具注册与路由分发）

```python
"""VMware MCP Server - Complete implementation with REST API, vmcli, and vmrun."""

import json
import os
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent

from .client import VMwareClient
from .vmcli import VMCli
from .vmrun import VMRun

server = Server("vmware-mcp")
_vm_path_cache: dict[str, str] = {}


def get_client() -> VMwareClient:
    return VMwareClient(
        host=os.getenv("VMWARE_HOST", "localhost"),
        port=int(os.getenv("VMWARE_PORT", "8697")),
        username=os.getenv("VMWARE_USERNAME", ""),
        password=os.getenv("VMWARE_PASSWORD", ""),
    )


def get_vmcli() -> VMCli:
    return VMCli()


def get_vmrun() -> VMRun:
    return VMRun()


async def get_vmx_path(vm_id: str) -> str:
    """Convert VM ID to vmx path. Supports both VM IDs and direct vmx paths."""
    if vm_id.endswith(".vmx") or "/" in vm_id or "\\" in vm_id:
        return vm_id

    if vm_id not in _vm_path_cache:
        client = get_client()
        vms = await client.list_vms()
        for vm in vms:
            _vm_path_cache[vm["id"]] = vm["path"]
    return _vm_path_cache.get(vm_id, "")


def T(name: str, desc: str, props: dict, required: list | None = None) -> Tool:
    """Helper to create Tool definitions."""
    schema = {"type": "object", "properties": props}
    if required:
        schema["required"] = required
    return Tool(name=name, description=desc, inputSchema=schema)
```

```
@server.list_tools()
async def list_tools() -> list[Tool]:
    return [
        # ==================== REST API ====================
        T("vm_list", "List all VMs", {}),
        T("vm_get", "Get VM settings", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("vm_create", "Clone a VM (REST)", {"vm_id": {"type": "string"}, "name":
{"type": "string"}}, ["vm_id", "name"]),
        T("vm_delete", "Delete a VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("vm_update", "Update VM settings", {"vm_id": {"type": "string"}, "cpu":
{"type": "integer"}, "memory": {"type": "integer"}}, ["vm_id"]),
        T("vm_power_get", "Get VM power state", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("vm_power_set", "Change VM power state", {"vm_id": {"type": "string"},
"state": {"type": "string", "enum": ["on", "off", "shutdown", "suspend", "pause",
"unpause"]}}, ["vm_id", "state"]),
        T("vm_nic_list", "List VM network adapters", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("vm_nic_create", "Create VM network adapter", {"vm_id": {"type":
"string"}, "type": {"type": "string", "enum": ["bridged", "nat", "hostonly",
"custom"]}}, ["vm_id", "type"]),
        T("vm_nic_delete", "Delete VM network adapter", {"vm_id": {"type":
"string"}, "index": {"type": "integer"}}, ["vm_id", "index"]),
        T("vm_ip_get", "Get VM IP address (REST)", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("vm_folder_list", "List VM shared folders", {"vm_id": {"type":
"string"}}, ["vm_id"]),
        T("vm_folder_create", "Create VM shared folder", {"vm_id": {"type":
"string"}, "folder_id": {"type": "string"}, "host_path": {"type": "string"},
"flags": {"type": "integer"}}, ["vm_id", "folder_id", "host_path"]),
        T("vm_folder_delete", "Delete VM shared folder", {"vm_id": {"type":
"string"}, "folder_id": {"type": "string"}}, ["vm_id", "folder_id"]),
        T("network_list", "List host virtual networks", {}),
        T("network_create", "Create host virtual network", {"name": {"type":
"string"}, "type": {"type": "string", "enum": ["bridged", "nat", "hostonly"]}},
["name", "type"]),
        T("network_portforward_list", "List port forwards", {"vmnet": {"type":
"string"}}, ["vmnet"]),
        T("network_portforward_set", "Set port forward", {"vmnet": {"type":
"string"}, "protocol": {"type": "string", "enum": ["tcp", "udp"]}, "port": {"type":
"integer"}, "guest_ip": {"type": "string"}, "guest_port": {"type": "integer"}},
["vmnet", "protocol", "port", "guest_ip", "guest_port"]),
        T("network_portforward_delete", "Delete port forward", {"vmnet": {"type":
"string"}, "protocol": {"type": "string"}, "port": {"type": "integer"}}, ["vmnet",
"protocol", "port"]),

        # ==================== VMRUN ====================
        T("vmrun_list", "List all running VMs", {}),
        T("vmrun_clone", "Clone VM (full/linked)", {"vm_id": {"type": "string"},
"dest_path": {"type": "string"}, "clone_type": {"type": "string", "enum": ["full",
"linked"]}, "snapshot": {"type": "string"}, "clone_name": {"type": "string"}},
["vm_id", "dest_path"]),
        T("vmrun_upgrade", "Upgrade VM format", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("vmrun_delete", "Delete VM (vmrun)", {"vm_id": {"type": "string"}},
```

```
["vm_id"]),
        T("vmrun_start", "Start VM", {"vm_id": {"type": "string"}, "gui": {"type":
"boolean"}}, ["vm_id"]),
        T("vmrun_stop", "Stop VM", {"vm_id": {"type": "string"}, "hard": {"type":
"boolean"}}, ["vm_id"]),
        T("vmrun_reset", "Reset VM", {"vm_id": {"type": "string"}, "hard": {"type":
"boolean"}}, ["vm_id"]),
        T("vmrun_suspend", "Suspend VM", {"vm_id": {"type": "string"}, "hard":
{"type": "boolean"}}, ["vm_id"]),
        T("vmrun_pause", "Pause VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("vmrun_unpause", "Unpause VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("vmrun_snapshot_list", "List snapshots (tree)", {"vm_id": {"type":
"string"}, "show_tree": {"type": "boolean"}}, ["vm_id"]),
        T("vmrun_snapshot_take", "Take snapshot", {"vm_id": {"type": "string"},
"name": {"type": "string"}}, ["vm_id", "name"]),
        T("vmrun_snapshot_delete", "Delete snapshot", {"vm_id": {"type": "string"},
"name": {"type": "string"}, "delete_children": {"type": "boolean"}}, ["vm_id",
"name"]),
        T("vmrun_snapshot_revert", "Revert to snapshot", {"vm_id": {"type":
"string"}, "name": {"type": "string"}}, ["vm_id", "name"]),
        T("vmrun_file_exists", "Check if file exists in guest", {"vm_id": {"type":
"string"}, "path": {"type": "string"}, "user": {"type": "string"}, "password":
{"type": "string"}}, ["vm_id", "path"]),
        T("vmrun_dir_exists", "Check if directory exists in guest", {"vm_id":
{"type": "string"}, "path": {"type": "string"}, "user": {"type": "string"},
"password": {"type": "string"}}, ["vm_id", "path"]),
        T("vmrun_ls", "List directory in guest", {"vm_id": {"type": "string"},
"path": {"type": "string"}, "user": {"type": "string"}, "password": {"type":
"string"}}, ["vm_id", "path"]),
        T("vmrun_mkdir", "Create directory in guest", {"vm_id": {"type": "string"},
"path": {"type": "string"}, "user": {"type": "string"}, "password": {"type":
"string"}}, ["vm_id", "path"]),
        T("vmrun_rmdir", "Delete directory in guest", {"vm_id": {"type": "string"},
"path": {"type": "string"}, "user": {"type": "string"}, "password": {"type":
"string"}}, ["vm_id", "path"]),
        T("vmrun_rm", "Delete file in guest", {"vm_id": {"type": "string"}, "path":
{"type": "string"}, "user": {"type": "string"}, "password": {"type": "string"}},
["vm_id", "path"]),
        T("vmrun_rename", "Rename file in guest", {"vm_id": {"type": "string"},
"old_path": {"type": "string"}, "new_path": {"type": "string"}, "user": {"type":
"string"}, "password": {"type": "string"}}, ["vm_id", "old_path", "new_path"]),
        T("vmrun_copy_to", "Copy file from host to guest", {"vm_id": {"type":
"string"}, "host_path": {"type": "string"}, "guest_path": {"type": "string"},
"user": {"type": "string"}, "password": {"type": "string"}}, ["vm_id", "host_path",
"guest_path"]),
        T("vmrun_copy_from", "Copy file from guest to host", {"vm_id": {"type":
"string"}, "guest_path": {"type": "string"}, "host_path": {"type": "string"},
"user": {"type": "string"}, "password": {"type": "string"}}, ["vm_id",
"guest_path", "host_path"]),
        T("vmrun_temp_file", "Create temp file in guest", {"vm_id": {"type":
"string"}, "user": {"type": "string"}, "password": {"type": "string"}}, ["vm_id"]),
        T("vmrun_run", "Run program in guest", {"vm_id": {"type": "string"},
"program": {"type": "string"}, "args": {"type": "string"}, "no_wait": {"type":
"boolean"}, "interactive": {"type": "boolean"}, "user": {"type": "string"},
"password": {"type": "string"}}, ["vm_id", "program"]),
        T("vmrun_script", "Run script in guest", {"vm_id": {"type": "string"},
```

```
"interpreter": {"type": "string"}, "script": {"type": "string"}, "no_wait":
{"type": "boolean"}, "user": {"type": "string"}, "password": {"type": "string"}},
["vm_id", "interpreter", "script"]),
        T("vmrun_ps", "List processes in guest", {"vm_id": {"type": "string"},
"user": {"type": "string"}, "password": {"type": "string"}}, ["vm_id"]),
        T("vmrun_kill", "Kill process in guest", {"vm_id": {"type": "string"},
"pid": {"type": "integer"}, "user": {"type": "string"}, "password": {"type":
"string"}}, ["vm_id", "pid"]),
        T("vmrun_shared_enable", "Enable shared folders", {"vm_id": {"type":
"string"}}, ["vm_id"]),
        T("vmrun_shared_disable", "Disable shared folders", {"vm_id": {"type":
"string"}}, ["vm_id"]),
        T("vmrun_shared_add", "Add shared folder", {"vm_id": {"type": "string"},
"name": {"type": "string"}, "host_path": {"type": "string"}}, ["vm_id", "name",
"host_path"]),
        T("vmrun_shared_remove", "Remove shared folder", {"vm_id": {"type":
"string"}, "name": {"type": "string"}}, ["vm_id", "name"]),
        T("vmrun_shared_set", "Set shared folder state", {"vm_id": {"type":
"string"}, "name": {"type": "string"}, "host_path": {"type": "string"}, "writable":
{"type": "boolean"}}, ["vm_id", "name", "host_path"]),
        T("vmrun_device_connect", "Connect device", {"vm_id": {"type": "string"},
"device": {"type": "string"}}, ["vm_id", "device"]),
        T("vmrun_device_disconnect", "Disconnect device", {"vm_id": {"type":
"string"}, "device": {"type": "string"}}, ["vm_id", "device"]),
        T("vmrun_var_read", "Read VM variable", {"vm_id": {"type": "string"},
"var_type": {"type": "string", "enum": ["runtimeConfig", "guestEnv", "guestVar"]},
"name": {"type": "string"}, "user": {"type": "string"}, "password": {"type":
"string"}}, ["vm_id", "var_type", "name"]),
        T("vmrun_var_write", "Write VM variable", {"vm_id": {"type": "string"},
"var_type": {"type": "string", "enum": ["runtimeConfig", "guestEnv", "guestVar"]},
"name": {"type": "string"}, "value": {"type": "string"}, "user": {"type":
"string"}, "password": {"type": "string"}}, ["vm_id", "var_type", "name",
"value"]),
        T("vmrun_screenshot", "Capture VM screenshot", {"vm_id": {"type":
"string"}, "output_path": {"type": "string"}}, ["vm_id", "output_path"]),
        T("vmrun_keystrokes", "Type keystrokes in guest", {"vm_id": {"type":
"string"}, "keystrokes": {"type": "string"}}, ["vm_id", "keystrokes"]),
        T("vmrun_tools_install", "Install VMware Tools", {"vm_id": {"type":
"string"}}, ["vm_id"]),
        T("vmrun_tools_state", "Check VMware Tools state", {"vm_id": {"type":
"string"}}, ["vm_id"]),
        T("vmrun_guest_ip", "Get guest IP address", {"vm_id": {"type": "string"},
"wait": {"type": "boolean"}}, ["vm_id"]),
        T("vmrun_host_networks", "List host networks", {}),
        T("vmrun_portforward_list", "List port forwardings", {"network": {"type":
"string"}}, ["network"]),
        T("vmrun_portforward_set", "Set port forwarding", {"network": {"type":
"string"}, "protocol": {"type": "string"}, "host_port": {"type": "integer"},
"guest_ip": {"type": "string"}, "guest_port": {"type": "integer"}, "description":
{"type": "string"}}, ["network", "protocol", "host_port", "guest_ip",
"guest_port"]),
        T("vmrun_portforward_delete", "Delete port forwarding", {"network":
{"type": "string"}, "protocol": {"type": "string"}, "host_port": {"type":
"integer"}}, ["network", "protocol", "host_port"]),

        # ==================== VMCLI ====================
```

```
        T("snapshot_list", "List snapshots (vmcli)", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("snapshot_take", "Take snapshot (vmcli)", {"vm_id": {"type": "string"},
"name": {"type": "string"}}, ["vm_id", "name"]),
        T("snapshot_revert", "Revert to snapshot (vmcli)", {"vm_id": {"type":
"string"}, "name": {"type": "string"}}, ["vm_id", "name"]),
        T("snapshot_delete", "Delete snapshot (vmcli)", {"vm_id": {"type":
"string"}, "name": {"type": "string"}, "delete_children": {"type": "boolean"}},
["vm_id", "name"]),
        T("snapshot_clone", "Clone from snapshot", {"vm_id": {"type": "string"},
"snapshot_name": {"type": "string"}, "dest_path": {"type": "string"}, "clone_type":
{"type": "string", "enum": ["linked", "full"]}}, ["vm_id", "snapshot_name",
"dest_path"]),
        T("guest_run", "Run program in guest", {"vm_id": {"type": "string"},
"program": {"type": "string"}, "args": {"type": "string"}, "user": {"type":
"string"}, "password": {"type": "string"}}, ["vm_id", "program"]),
        T("guest_ps", "List processes", {"vm_id": {"type": "string"}, "user":
{"type": "string"}, "password": {"type": "string"}}, ["vm_id"]),
        T("guest_kill", "Kill process", {"vm_id": {"type": "string"}, "pid":
{"type": "integer"}, "user": {"type": "string"}, "password": {"type": "string"}},
["vm_id", "pid"]),
        T("guest_ls", "List files", {"vm_id": {"type": "string"}, "path": {"type":
"string"}, "user": {"type": "string"}, "password": {"type": "string"}}, ["vm_id",
"path"]),
        T("guest_mkdir", "Create directory", {"vm_id": {"type": "string"}, "path":
{"type": "string"}, "user": {"type": "string"}, "password": {"type": "string"}},
["vm_id", "path"]),
        T("guest_rm", "Delete file", {"vm_id": {"type": "string"}, "path": {"type":
"string"}, "user": {"type": "string"}, "password": {"type": "string"}}, ["vm_id",
"path"]),
        T("guest_rmdir", "Delete directory", {"vm_id": {"type": "string"}, "path":
{"type": "string"}, "user": {"type": "string"}, "password": {"type": "string"}},
["vm_id", "path"]),
        T("guest_copy_to", "Copy to guest", {"vm_id": {"type": "string"},
"host_path": {"type": "string"}, "guest_path": {"type": "string"}, "user": {"type":
"string"}, "password": {"type": "string"}}, ["vm_id", "host_path", "guest_path"]),
        T("guest_copy_from", "Copy from guest", {"vm_id": {"type": "string"},
"guest_path": {"type": "string"}, "host_path": {"type": "string"}, "user": {"type":
"string"}, "password": {"type": "string"}}, ["vm_id", "guest_path", "host_path"]),
        T("guest_env", "Get environment", {"vm_id": {"type": "string"}, "user":
{"type": "string"}, "password": {"type": "string"}}, ["vm_id"]),
        T("mks_screenshot", "Capture screenshot", {"vm_id": {"type": "string"},
"output_path": {"type": "string"}}, ["vm_id", "output_path"]),
        T("mks_send_key", "Send key sequence", {"vm_id": {"type": "string"},
"key_sequence": {"type": "string"}}, ["vm_id", "key_sequence"]),
        T("mks_query", "Query MKS state", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("chipset_query", "Query chipset config", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("chipset_set_cpu", "Set CPU count", {"vm_id": {"type": "string"},
"count": {"type": "integer"}}, ["vm_id", "count"]),
        T("chipset_set_memory", "Set memory (MB)", {"vm_id": {"type": "string"},
"size_mb": {"type": "integer"}}, ["vm_id", "size_mb"]),
        T("chipset_set_cores", "Set cores per socket", {"vm_id": {"type":
"string"}, "cores": {"type": "integer"}}, ["vm_id", "cores"]),
        T("tools_query", "Query Tools status", {"vm_id": {"type": "string"}},
```

```
["vm_id"]),
        T("tools_install", "Install Tools", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("tools_upgrade", "Upgrade Tools", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("template_create", "Create template", {"vm_id": {"type": "string"},
"template_path": {"type": "string"}, "name": {"type": "string"}}, ["vm_id",
"template_path", "name"]),
        T("template_deploy", "Deploy template", {"template_path": {"type":
"string"}, "dest_path": {"type": "string"}, "name": {"type": "string"}},
["template_path", "dest_path", "name"]),
        T("disk_query", "Query disk config", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("disk_create", "Create disk", {"vm_id": {"type": "string"}, "size_gb":
{"type": "integer"}, "disk_type": {"type": "string"}, "adapter": {"type":
"integer"}, "device": {"type": "integer"}}, ["vm_id", "size_gb"]),
        T("disk_extend", "Extend disk", {"vm_id": {"type": "string"},
"new_size_gb": {"type": "integer"}, "adapter": {"type": "integer"}, "device":
{"type": "integer"}}, ["vm_id", "new_size_gb"]),
        T("config_query", "Query config params", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("config_set", "Set config param", {"vm_id": {"type": "string"}, "key":
{"type": "string"}, "value": {"type": "string"}}, ["vm_id", "key", "value"]),
        T("power_query", "Query power state", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("power_start", "Start VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("power_stop", "Stop VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("power_pause", "Pause VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("power_unpause", "Unpause VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("power_reset", "Reset VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("power_suspend", "Suspend VM", {"vm_id": {"type": "string"}}, ["vm_id"]),
        T("ethernet_query", "Query ethernet config", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("ethernet_set_type", "Set connection type", {"vm_id": {"type": "string"},
"index": {"type": "integer"}, "type": {"type": "string", "enum": ["bridged", "nat",
"hostonly", "custom"]}}, ["vm_id", "index", "type"]),
        T("ethernet_set_present", "Set ethernet present", {"vm_id": {"type":
"string"}, "index": {"type": "integer"}, "present": {"type": "boolean"}}, ["vm_id",
"index", "present"]),
        T("ethernet_set_connected", "Set start connected", {"vm_id": {"type":
"string"}, "index": {"type": "integer"}, "connected": {"type": "boolean"}},
["vm_id", "index", "connected"]),
        T("ethernet_set_device", "Set virtual device", {"vm_id": {"type":
"string"}, "index": {"type": "integer"}, "device": {"type": "string"}}, ["vm_id",
"index", "device"]),
        T("ethernet_set_network", "Set network name", {"vm_id": {"type": "string"},
"index": {"type": "integer"}, "name": {"type": "string"}}, ["vm_id", "index",
"name"]),
        T("ethernet_purge", "Remove ethernet adapter", {"vm_id": {"type":
"string"}, "index": {"type": "integer"}}, ["vm_id", "index"]),
        T("hgfs_query", "Query shared folders", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("hgfs_set_enabled", "Enable/disable share", {"vm_id": {"type": "string"},
"index": {"type": "integer"}, "enabled": {"type": "boolean"}}, ["vm_id", "index",
"enabled"]),
        T("hgfs_set_path", "Set host path", {"vm_id": {"type": "string"}, "index":
```

```
{"type": "integer"}, "path": {"type": "string"}}, ["vm_id", "index", "path"]),
        T("hgfs_set_name", "Set guest name", {"vm_id": {"type": "string"}, "index":
{"type": "integer"}, "name": {"type": "string"}}, ["vm_id", "index", "name"]),
        T("hgfs_set_read", "Set read access", {"vm_id": {"type": "string"},
"index": {"type": "integer"}, "read": {"type": "boolean"}}, ["vm_id", "index",
"read"]),
        T("hgfs_set_write", "Set write access", {"vm_id": {"type": "string"},
"index": {"type": "integer"}, "write": {"type": "boolean"}}, ["vm_id", "index",
"write"]),
        T("serial_query", "Query serial ports", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("serial_set_present", "Set serial present", {"vm_id": {"type": "string"},
"index": {"type": "integer"}, "present": {"type": "boolean"}}, ["vm_id", "index",
"present"]),
        T("serial_purge", "Remove serial port", {"vm_id": {"type": "string"},
"index": {"type": "integer"}}, ["vm_id", "index"]),
        T("sata_query", "Query SATA config", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("sata_set_present", "Set SATA present", {"vm_id": {"type": "string"},
"adapter": {"type": "integer"}, "present": {"type": "boolean"}}, ["vm_id",
"adapter", "present"]),
        T("sata_purge", "Remove SATA adapter", {"vm_id": {"type": "string"},
"adapter": {"type": "integer"}}, ["vm_id", "adapter"]),
        T("nvme_query", "Query NVMe config", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("nvme_set_present", "Set NVMe present", {"vm_id": {"type": "string"},
"adapter": {"type": "integer"}, "present": {"type": "boolean"}}, ["vm_id",
"adapter", "present"]),
        T("nvme_purge", "Remove NVMe adapter", {"vm_id": {"type": "string"},
"adapter": {"type": "integer"}}, ["vm_id", "adapter"]),
        T("vprobes_query", "Query VProbes", {"vm_id": {"type": "string"}},
["vm_id"]),
        T("vprobes_enable", "Enable VProbes", {"vm_id": {"type": "string"},
"enabled": {"type": "boolean"}}, ["vm_id", "enabled"]),
        T("vprobes_load", "Load VProbes script", {"vm_id": {"type": "string"},
"script_path": {"type": "string"}}, ["vm_id", "script_path"]),
        T("vprobes_reset", "Reset VProbes", {"vm_id": {"type": "string"}},
["vm_id"]),
    ]


@server.call_tool()
async def call_tool(name: str, arguments: dict) -> list[TextContent]:
    client = get_client()
    vmcli = get_vmcli()
    vmrun = get_vmrun()
    result = None
    a = arguments

    async def vmx(vm_id: str) -> str:
        return await get_vmx_path(vm_id)

    # ==================== REST API ====================
    if name == "vm_list":
        result = await client.list_vms()
        for vm in result:
```

```
            _vm_path_cache[vm["id"]] = vm["path"]
    elif name == "vm_get":
        result = await client.get_vm(a["vm_id"])
    elif name == "vm_create":
        result = await client.create_vm(a["vm_id"], a["name"])
    elif name == "vm_delete":
        await client.delete_vm(a["vm_id"])
        result = {"status": "deleted"}
    elif name == "vm_update":
        settings = {k: v for k, v in a.items() if k != "vm_id" and v is not None}
        result = await client.update_vm(a["vm_id"], settings)
    elif name == "vm_power_get":
        result = await client.get_power_state(a["vm_id"])
    elif name == "vm_power_set":
        result = await client.change_power_state(a["vm_id"], a["state"])
    elif name == "vm_nic_list":
        result = await client.list_nics(a["vm_id"])
    elif name == "vm_nic_create":
        result = await client.create_nic(a["vm_id"], {"type": a["type"]})
    elif name == "vm_nic_delete":
        await client.delete_nic(a["vm_id"], a["index"])
        result = {"status": "deleted"}
    elif name == "vm_ip_get":
        result = await client.get_vm_ip(a["vm_id"])
    elif name == "vm_folder_list":
        result = await client.list_shared_folders(a["vm_id"])
    elif name == "vm_folder_create":
        result = await client.create_shared_folder(a["vm_id"], {"folder_id":
a["folder_id"], "host_path": a["host_path"], "flags": a.get("flags", 0)})
    elif name == "vm_folder_delete":
        await client.delete_shared_folder(a["vm_id"], a["folder_id"])
        result = {"status": "deleted"}
    elif name == "network_list":
        result = await client.list_networks()
    elif name == "network_create":
        result = await client.create_network({"name": a["name"], "type":
a["type"]})
    elif name == "network_portforward_list":
        result = await client.get_portforwards(a["vmnet"])
    elif name == "network_portforward_set":
        result = await client.update_portforward(a["vmnet"], a["protocol"],
a["port"], {"guestIp": a["guest_ip"], "guestPort": a["guest_port"]})
    elif name == "network_portforward_delete":
        await client.delete_portforward(a["vmnet"], a["protocol"], a["port"])
        result = {"status": "deleted"}

    # ==================== VMRUN ====================
    elif name == "vmrun_list":
        result = await vmrun.list_running()
    elif name == "vmrun_clone":
        result = await vmrun.clone(await vmx(a["vm_id"]), a["dest_path"],
a.get("clone_type", "linked"), a.get("snapshot", ""), a.get("clone_name", ""))
    elif name == "vmrun_upgrade":
        result = await vmrun.upgrade_vm(await vmx(a["vm_id"]))
    elif name == "vmrun_delete":
        result = await vmrun.delete_vm(await vmx(a["vm_id"]))
```

```
    elif name == "vmrun_start":
        result = await vmrun.start(await vmx(a["vm_id"]), a.get("gui", True))
    elif name == "vmrun_stop":
        result = await vmrun.stop(await vmx(a["vm_id"]), a.get("hard", False))
    elif name == "vmrun_reset":
        result = await vmrun.reset(await vmx(a["vm_id"]), a.get("hard", False))
    elif name == "vmrun_suspend":
        result = await vmrun.suspend(await vmx(a["vm_id"]), a.get("hard", False))
    elif name == "vmrun_pause":
        result = await vmrun.pause(await vmx(a["vm_id"]))
    elif name == "vmrun_unpause":
        result = await vmrun.unpause(await vmx(a["vm_id"]))
    elif name == "vmrun_snapshot_list":
        result = await vmrun.list_snapshots(await vmx(a["vm_id"]),
a.get("show_tree", False))
    elif name == "vmrun_snapshot_take":
        result = await vmrun.snapshot(await vmx(a["vm_id"]), a["name"])
    elif name == "vmrun_snapshot_delete":
        result = await vmrun.delete_snapshot(await vmx(a["vm_id"]), a["name"],
a.get("delete_children", False))
    elif name == "vmrun_snapshot_revert":
        result = await vmrun.revert_to_snapshot(await vmx(a["vm_id"]), a["name"])
    elif name == "vmrun_file_exists":
        result = await vmrun.file_exists(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_dir_exists":
        result = await vmrun.directory_exists(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_ls":
        result = await vmrun.list_directory(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_mkdir":
        result = await vmrun.create_directory(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_rmdir":
        result = await vmrun.delete_directory(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_rm":
        result = await vmrun.delete_file(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_rename":
        result = await vmrun.rename_file(await vmx(a["vm_id"]), a["old_path"],
a["new_path"], a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_copy_to":
        result = await vmrun.copy_to_guest(await vmx(a["vm_id"]), a["host_path"],
a["guest_path"], a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_copy_from":
        result = await vmrun.copy_from_guest(await vmx(a["vm_id"]),
a["guest_path"], a["host_path"], a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_temp_file":
        result = await vmrun.create_temp_file(await vmx(a["vm_id"]), a.get("user",
""), a.get("password", ""))
    elif name == "vmrun_run":
        result = await vmrun.run_program(await vmx(a["vm_id"]), a["program"],
a.get("args", ""), a.get("no_wait", False), False, a.get("interactive", False),
a.get("user", ""), a.get("password", ""))
```

```
    elif name == "vmrun_script":
        result = await vmrun.run_script(await vmx(a["vm_id"]), a["interpreter"],
a["script"], a.get("no_wait", False), False, False, a.get("user", ""),
a.get("password", ""))
    elif name == "vmrun_ps":
        result = await vmrun.list_processes(await vmx(a["vm_id"]), a.get("user",
""), a.get("password", ""))
    elif name == "vmrun_kill":
        result = await vmrun.kill_process(await vmx(a["vm_id"]), a["pid"],
a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_shared_enable":
        result = await vmrun.enable_shared_folders(await vmx(a["vm_id"]))
    elif name == "vmrun_shared_disable":
        result = await vmrun.disable_shared_folders(await vmx(a["vm_id"]))
    elif name == "vmrun_shared_add":
        result = await vmrun.add_shared_folder(await vmx(a["vm_id"]), a["name"],
a["host_path"])
    elif name == "vmrun_shared_remove":
        result = await vmrun.remove_shared_folder(await vmx(a["vm_id"]), a["name"])
    elif name == "vmrun_shared_set":
        result = await vmrun.set_shared_folder_state(await vmx(a["vm_id"]),
a["name"], a["host_path"], a.get("writable", True))
    elif name == "vmrun_device_connect":
        result = await vmrun.connect_device(await vmx(a["vm_id"]), a["device"])
    elif name == "vmrun_device_disconnect":
        result = await vmrun.disconnect_device(await vmx(a["vm_id"]), a["device"])
    elif name == "vmrun_var_read":
        result = await vmrun.read_variable(await vmx(a["vm_id"]), a["var_type"],
a["name"], a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_var_write":
        result = await vmrun.write_variable(await vmx(a["vm_id"]), a["var_type"],
a["name"], a["value"], a.get("user", ""), a.get("password", ""))
    elif name == "vmrun_screenshot":
        result = await vmrun.capture_screen(await vmx(a["vm_id"]),
a["output_path"])
    elif name == "vmrun_keystrokes":
        result = await vmrun.type_keystrokes(await vmx(a["vm_id"]),
a["keystrokes"])
    elif name == "vmrun_tools_install":
        result = await vmrun.install_tools(await vmx(a["vm_id"]))
    elif name == "vmrun_tools_state":
        result = await vmrun.check_tools_state(await vmx(a["vm_id"]))
    elif name == "vmrun_guest_ip":
        result = await vmrun.get_guest_ip(await vmx(a["vm_id"]), a.get("wait",
False))
    elif name == "vmrun_host_networks":
        result = await vmrun.list_host_networks()
    elif name == "vmrun_portforward_list":
        result = await vmrun.list_port_forwardings(a["network"])
    elif name == "vmrun_portforward_set":
        result = await vmrun.set_port_forwarding(a["network"], a["protocol"],
a["host_port"], a["guest_ip"], a["guest_port"], a.get("description", ""))
    elif name == "vmrun_portforward_delete":
        result = await vmrun.delete_port_forwarding(a["network"], a["protocol"],
a["host_port"])
```

```
    # ==================== VMCLI ====================
    elif name == "snapshot_list":
        result = await vmcli.snapshot_list(await vmx(a["vm_id"]))
    elif name == "snapshot_take":
        result = await vmcli.snapshot_take(await vmx(a["vm_id"]), a["name"])
    elif name == "snapshot_revert":
        result = await vmcli.snapshot_revert(await vmx(a["vm_id"]), a["name"])
    elif name == "snapshot_delete":
        result = await vmcli.snapshot_delete(await vmx(a["vm_id"]), a["name"],
a.get("delete_children", False))
    elif name == "snapshot_clone":
        result = await vmcli.snapshot_clone(await vmx(a["vm_id"]),
a["snapshot_name"], a["dest_path"], a.get("clone_type", "linked"))
    elif name == "guest_run":
        result = await vmcli.guest_run(await vmx(a["vm_id"]), a["program"],
a.get("args", ""), a.get("user", ""), a.get("password", ""))
    elif name == "guest_ps":
        result = await vmcli.guest_ps(await vmx(a["vm_id"]), a.get("user", ""),
a.get("password", ""))
    elif name == "guest_kill":
        result = await vmcli.guest_kill(await vmx(a["vm_id"]), a["pid"],
a.get("user", ""), a.get("password", ""))
    elif name == "guest_ls":
        result = await vmcli.guest_ls(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "guest_mkdir":
        result = await vmcli.guest_mkdir(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "guest_rm":
        result = await vmcli.guest_rm(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "guest_rmdir":
        result = await vmcli.guest_rmdir(await vmx(a["vm_id"]), a["path"],
a.get("user", ""), a.get("password", ""))
    elif name == "guest_copy_to":
        result = await vmcli.guest_copy_to(await vmx(a["vm_id"]), a["host_path"],
a["guest_path"], a.get("user", ""), a.get("password", ""))
    elif name == "guest_copy_from":
        result = await vmcli.guest_copy_from(await vmx(a["vm_id"]),
a["guest_path"], a["host_path"], a.get("user", ""), a.get("password", ""))
    elif name == "guest_env":
        result = await vmcli.guest_env(await vmx(a["vm_id"]), a.get("user", ""),
a.get("password", ""))
    elif name == "mks_screenshot":
        result = await vmcli.mks_screenshot(await vmx(a["vm_id"]),
a["output_path"])
    elif name == "mks_send_key":
        result = await vmcli.mks_send_key(await vmx(a["vm_id"]), a["key_sequence"])
    elif name == "mks_query":
        result = await vmcli.mks_query(await vmx(a["vm_id"]))
    elif name == "chipset_query":
        result = await vmcli.chipset_query(await vmx(a["vm_id"]))
    elif name == "chipset_set_cpu":
        result = await vmcli.chipset_set_cpu(await vmx(a["vm_id"]), a["count"])
    elif name == "chipset_set_memory":
        result = await vmcli.chipset_set_memory(await vmx(a["vm_id"]),
```

```
a["size_mb"])
    elif name == "chipset_set_cores":
        result = await vmcli.chipset_set_cores_per_socket(await vmx(a["vm_id"]),
a["cores"])
    elif name == "tools_query":
        result = await vmcli.tools_query(await vmx(a["vm_id"]))
    elif name == "tools_install":
        result = await vmcli.tools_install(await vmx(a["vm_id"]))
    elif name == "tools_upgrade":
        result = await vmcli.tools_upgrade(await vmx(a["vm_id"]))
    elif name == "template_create":
        result = await vmcli.template_create(await vmx(a["vm_id"]),
a["template_path"], a["name"])
    elif name == "template_deploy":
        result = await vmcli.template_deploy(a["template_path"], a["dest_path"],
a["name"])
    elif name == "disk_query":
        result = await vmcli.disk_query(await vmx(a["vm_id"]))
    elif name == "disk_create":
        result = await vmcli.disk_create(await vmx(a["vm_id"]), a["size_gb"],
a.get("disk_type", "scsi"), a.get("adapter", 0), a.get("device", 0))
    elif name == "disk_extend":
        result = await vmcli.disk_extend(await vmx(a["vm_id"]), a["new_size_gb"],
a.get("adapter", 0), a.get("device", 0))
    elif name == "config_query":
        result = await vmcli.config_query(await vmx(a["vm_id"]))
    elif name == "config_set":
        result = await vmcli.config_set(await vmx(a["vm_id"]), a["key"],
a["value"])
    elif name == "power_query":
        result = await vmcli.power_query(await vmx(a["vm_id"]))
    elif name == "power_start":
        result = await vmcli.power_start(await vmx(a["vm_id"]))
    elif name == "power_stop":
        result = await vmcli.power_stop(await vmx(a["vm_id"]))
    elif name == "power_pause":
        result = await vmcli.power_pause(await vmx(a["vm_id"]))
    elif name == "power_unpause":
        result = await vmcli.power_unpause(await vmx(a["vm_id"]))
    elif name == "power_reset":
        result = await vmcli.power_reset(await vmx(a["vm_id"]))
    elif name == "power_suspend":
        result = await vmcli.power_suspend(await vmx(a["vm_id"]))
    elif name == "ethernet_query":
        result = await vmcli.ethernet_query(await vmx(a["vm_id"]))
    elif name == "ethernet_set_type":
        result = await vmcli.ethernet_set_connection_type(await vmx(a["vm_id"]),
a["index"], a["type"])
    elif name == "ethernet_set_present":
        result = await vmcli.ethernet_set_present(await vmx(a["vm_id"]),
a["index"], a["present"])
    elif name == "ethernet_set_connected":
        result = await vmcli.ethernet_set_start_connected(await vmx(a["vm_id"]),
a["index"], a["connected"])
    elif name == "ethernet_set_device":
        result = await vmcli.ethernet_set_virtual_device(await vmx(a["vm_id"]),
```

```
a["index"], a["device"])
    elif name == "ethernet_set_network":
        result = await vmcli.ethernet_set_network_name(await vmx(a["vm_id"]),
a["index"], a["name"])
    elif name == "ethernet_purge":
        result = await vmcli.ethernet_purge(await vmx(a["vm_id"]), a["index"])
    elif name == "hgfs_query":
        result = await vmcli.hgfs_query(await vmx(a["vm_id"]))
    elif name == "hgfs_set_enabled":
        result = await vmcli.hgfs_set_enabled(await vmx(a["vm_id"]), a["index"],
a["enabled"])
    elif name == "hgfs_set_path":
        result = await vmcli.hgfs_set_host_path(await vmx(a["vm_id"]), a["index"],
a["path"])
    elif name == "hgfs_set_name":
        result = await vmcli.hgfs_set_guest_name(await vmx(a["vm_id"]), a["index"],
a["name"])
    elif name == "hgfs_set_read":
        result = await vmcli.hgfs_set_read_access(await vmx(a["vm_id"]),
a["index"], a["read"])
    elif name == "hgfs_set_write":
        result = await vmcli.hgfs_set_write_access(await vmx(a["vm_id"]),
a["index"], a["write"])
    elif name == "serial_query":
        result = await vmcli.serial_query(await vmx(a["vm_id"]))
    elif name == "serial_set_present":
        result = await vmcli.serial_set_present(await vmx(a["vm_id"]), a["index"],
a["present"])
    elif name == "serial_purge":
        result = await vmcli.serial_purge(await vmx(a["vm_id"]), a["index"])
    elif name == "sata_query":
        result = await vmcli.sata_query(await vmx(a["vm_id"]))
    elif name == "sata_set_present":
        result = await vmcli.sata_set_present(await vmx(a["vm_id"]), a["adapter"],
a["present"])
    elif name == "sata_purge":
        result = await vmcli.sata_purge(await vmx(a["vm_id"]), a["adapter"])
    elif name == "nvme_query":
        result = await vmcli.nvme_query(await vmx(a["vm_id"]))
    elif name == "nvme_set_present":
        result = await vmcli.nvme_set_present(await vmx(a["vm_id"]), a["adapter"],
a["present"])
    elif name == "nvme_purge":
        result = await vmcli.nvme_purge(await vmx(a["vm_id"]), a["adapter"])
    elif name == "vprobes_query":
        result = await vmcli.vprobes_query(await vmx(a["vm_id"]))
    elif name == "vprobes_enable":
        result = await vmcli.vprobes_set_enabled(await vmx(a["vm_id"]),
a["enabled"])
    elif name == "vprobes_load":
        result = await vmcli.vprobes_load(await vmx(a["vm_id"]), a["script_path"])
    elif name == "vprobes_reset":
        result = await vmcli.vprobes_reset(await vmx(a["vm_id"]))

    if isinstance(result, str):
        return [TextContent(type="text", text=result if result else "OK")]
```

```
    return [TextContent(type="text", text=json.dumps(result, indent=2) if result
else "OK")]


def main():
    import asyncio

    async def run():
        async with stdio_server() as (read_stream, write_stream):
            await server.run(read_stream, write_stream,
server.create_initialization_options())

    asyncio.run(run())


if __name__ == "__main__":
    main()
```

5.3 启动脚本

`run-vmware-mcp.bat`：

```
@echo off
set VMWARE_USERNAME=<vmrest用户名>
set VMWARE_PASSWORD=<vmrest密码>
set VMWARE_HOST=localhost
set VMWARE_PORT=8697
set VMRUN_PATH=C:\Program Files\VMware\VMware Workstation\vmrun.exe
set VMCLI_PATH=C:\Program Files\VMware\VMware Workstation\vmcli.exe

"D:\opencode\vmware-mcp\.venv\Scripts\vmware-mcp.exe"
```

5.4 MCP Server 配置（添加至 opencode.json）

```
{
  "vmware-mcp": {
    "type": "local",
    "command": ["D:\\opencode\\vmware-mcp\\run-vmware-mcp.bat"],
    "enabled": true,
    "timeout": 60000
  }
}
```

配置为 `type: "local"` 后，AI Agent 框架会自动启动该服务，无需手动运行。

5.5 验证服务启动

```
# 使用项目虚拟环境启动
D:\opencode\vmware-mcp\.venv\Scripts\vmware-mcp.exe
```

正常启动后不会有任何输出——MCP 通过标准输入输出通信，AI Agent 框架会用 JSON-RPC 协议与它交互。

---

6. 解决核心难题：捕获 Guest 命令输出

6.1 问题本质

vmrun 的 `runProgramInGuest` 和 `runScriptInGuest` 只返回执行状态（成功/失败），**不返回命令在虚拟机中产生的 stdout。**

6.2 解决方案：文件式 IPC

原理很简单——既然 vmrun 不返回 stdout，那就手动把输出写到文件里，再用 vmrun 把文件取回来。

```
flowchart LR
    A[Step 1: 执行命令，重定向到 /tmp/_vout] --> B[Step 2: CopyFileFromGuestToHost 取回]
    B --> C[Step 3: 读取宿主机临时文件]
    C --> D[Step 4: 清理双方临时文件]
```

6.3 核心代码实现

```
# src/vmware_mcp/vmrun.py — _run_with_output 方法

async def _run_with_output(self, vmx_path: str, shell_cmd: str,
                           user: str = "", password: str = "") -> str:
    # Step 1: 执行命令，stdout+stderr 全部重定向到临时文件
    await self._run("runScriptInGuest", vmx_path, "/bin/sh",
                    f"{shell_cmd} > /tmp/_vout 2>&1",
                    guest_user=user, guest_pass=password)

    local_tmp = os.path.join(tempfile.gettempdir(), f"_vmrun_vout_{os.getpid()}")
    output = ""
    try:
        # Step 2: 从 guest 复制到宿主机
        try:
            await self._run("CopyFileFromGuestToHost", vmx_path,
                            "/tmp/_vout", local_tmp,
                            guest_user=user, guest_pass=password)
        except RuntimeError:
            pass  # 文件可能不存在（命令未产生输出）
        # Step 3: 读取宿主机文件
        if os.path.exists(local_tmp):
            with open(local_tmp, "r", encoding="utf-8", errors="replace") as f:
                output = f.read().strip()
    finally:
        # Step 4: 清理
        if os.path.exists(local_tmp):
            os.remove(local_tmp)
        await self._run("runScriptInGuest", vmx_path, "/bin/sh",
                        "rm -f /tmp/_vout",
                        guest_user=user, guest_pass=password)
    return output
```

6.4 使用方法

修复后，直接调用 MCP 工具的 `vmrun_run` 或 `vmrun_script` 即可得到命令输出：

执行程序：

| 参数 | 值 |

| --- | --- |

| vm_id | `D:\虚拟机\kali64\kali64.vmx` |

| program | `/usr/bin/whoami` |

| user | `<虚拟机用户名>` |

| password | `<虚拟机密码>` |

→ 返回：`<虚拟机用户名>`

执行脚本：

| 参数 | 值 |

| --- | --- |

| vm_id | `D:\虚拟机\kali64\kali64.vmx` |

| interpreter | `/bin/sh` |

| script | `free -h | head -3` |

| user | `<虚拟机用户名>` |

| password | `<虚拟机密码>` |

→ 返回：

```
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       1.0Gi       2.0Gi        18Mi       989Mi       2.8Gi
Swap:          975Mi          0B       975Mi
```

---

7. 日常操作指南

7.1 可用工具一览

命令执行

| 工具 | 说明 | 返回内容 |

| --- | --- | --- |

| `vmrun_run` | 执行程序 | ✅ stdout+stderr |

| `vmrun_script` | 执行脚本 | ✅ stdout+stderr |

| `vmrun_run` + `no_wait=true` | 后台执行，不等待 | ❌ 不返回 |

文件操作

| 工具 | 说明 |

| --- | --- |

| `vmrun_ls` | 列出目录内容 |

| `vmrun_copy_to` | 宿主机 → 虚拟机 |

| `vmrun_copy_from` | 虚拟机 → 宿主机 |

| `vmrun_mkdir` | 创建目录 |

| `vmrun_rm` | 删除文件/目录 |

| `vmrun_rename` | 重命名 |

| `vmrun_file_exists` | 检查文件是否存在 |

| `vmrun_temp_file` | 创建临时文件（备用方案使用） |

电源管理

| 工具 | 说明 |

| --- | --- |

| `vmrun_start` | 启动虚拟机 |

| `vmrun_stop` | 停止虚拟机 |

| `vmrun_suspend` | 挂起 |

| `vmrun_pause` | 暂停 |

| `vmrun_reset` | 重启 |

快照管理

| 工具 | 说明 |

| --- | --- |

| `vmrun_snapshot_take` | 创建快照 |

| `vmrun_snapshot_list` | 列出快照 |

| `vmrun_snapshot_revert` | 恢复到快照 |

| `vmrun_snapshot_delete` | 删除快照 |

其他

| 工具 | 说明 |

| --- | --- |

| `vmrun_screenshot` | 截取虚拟机屏幕 |

| `vmrun_guest_ip` | 获取虚拟机 IP |

| `mks_send_key` | 发送键盘按键 |

| `vmrun_keystrokes` | 模拟键盘输入 |

7.2 提权操作（Kali Linux）

Kali 默认禁用 root 账户，需要通过 sudo 提权：

```
# 语法：echo <虚拟机密码> | sudo -S <命令>

# 示例：以 root 身份读取文件
echo <虚拟机用户名> | sudo -S cat /root/secret.log

# 示例：以 root 身份创建文件
echo "no network" | sudo tee /root/hello.log > /dev/null
```

在 MCP 工具调用中：

| 参数 | 值 |

| --- | --- |

| script | `echo <虚拟机用户名> \| sudo -S cat /root/hello_kali.log` |

| user | `<虚拟机用户名>` |

| password | `<虚拟机密码>` |

执行结果：`[sudo] password for <虚拟机用户名>: no network`

> 注意：`[sudo] password for <虚拟机用户名>:` 提示文本是 stderr 输出，会被 `2>&1` 捕获。可以通过 `grep -v` 过滤。

7.3 完整操作示例

场景：排查磁盘空间

```
用户：帮我看看虚拟机磁盘使用情况

AI 内部流程：
1. 调用 vmrun_script → 执行 "df -h"
2. 分析输出 → 根分区使用率 88%
3. 调用 vmrun_script → 执行 "du -sh /var/log/* | sort -rh | head -5"
4. 分析输出 → 找出大文件
5. 总结报告给用户
```

场景：从宿主机传文件到虚拟机

```
用户：把 config.yaml 传到虚拟机的 /etc/ 目录

AI 内部流程：
1. 在宿主机创建配置文件
2. 调用 vmrun_copy_to → 复制到虚拟机
3. 调用 vmrun_script → 执行 "ls -la /etc/config.yaml" 验证
```

---

8. 安全边界分析

8.1 网络安全

```
内网视角看这套系统：

宿主机 → 内网交换机 → 其他内网设备
   │
   └── VMware Tools（宿主机内存通道）
         │
         └── 虚拟机

内网交换机能看到的流量：零
内网其他设备能感知到的：零
虚拟机的网络接口：完全静默
```

8.2 通信原理



8.3 安全优势

| 攻击面 | SSH 方案 | VMware Tools 方案 |

| --- | --- | --- |

| 网络嗅探 | 可嗅探到 SSH 流量 | ❌ 无网络流量 |

| 端口扫描 | 22 端口暴露 | ❌ 无端口暴露 |

| 爆破攻击 | SSHD 暴露在外 | ❌ 完全不可达 |

| MITM | TCP 连接可能被劫持 | ❌ 无网络连接 |

8.4 局限性与风险

| 风险点 | 说明 |

| --- | --- |

| VMware Tools 必须安装 | 没有 Tools 则 vmrun 无法工作 |

| 仅限 VMware 生态 | 不能用于 VirtualBox、Hyper-V |

| 单机限制 | vmrun 只能操作本机 VM，不能跨宿主机 |

| AI 误操作 | AI 生成的危险命令（如 `rm -rf`）需要加白名单过滤 |

| 凭据管理 | vmrun 需要 guest 凭据，需安全存储 |

---

9. 常见问题 Q&A

Q1: 虚拟机的网线拔了还能执行命令吗?

能。 vmrun 走的是 VMware Tools 的 VMCI 虚拟硬件通道，不经过物理网卡。只要 VMware Tools 在客户机中正常运行，即使 `systemctl stop networking`、拔掉网线、禁用网卡，vmrun 依然能执行命令并返回输出。

Q2: 为什么不用 SSH?

DeepSeek 推荐的 SSH 方案是通用的解决方案，适用于大多数远程管理场景。但在这个特定需求中——**虚拟机必须对内网完全静默**——SSH 的 TCP 流量会经过内网网卡，不符合安全要求。VMware Tools 通道是更优选择。

Q3: 为什么 vmrun 默认不返回命令输出?

这是 VMware 的设计选择。vmrun 的 `runProgramInGuest` 设计逻辑是"启动程序 → 等待退出 → 返回退出码"，它关注的是程序的退出状态而非输出内容。要获取输出，必须通过文件中转。

Q4: 文件式 IPC 安全吗？临时文件会不会被其他进程读到?

临时文件路径是 `/tmp/_vout`，仅在命令执行期间存在（毫秒级），执行完毕后会被自动清理。在 Kali Linux 中 `/tmp` 目录的权限是 `drwxrwxrwt`（粘滞位），其他用户不能删除/修改不属于自己的文件。风险极低。

Q5: 如果命令执行失败怎么办?

vmrun 会返回非零退出码，`_run` 方法会抛出 `RuntimeError`。`_run_with_output` 中如果 `CopyFileFromGuestToHost` 失败（文件不存在），不会抛异常，而是返回空字符串。

Q6: 如果命令没有输出（例如 `touch`）会怎样?

临时文件依然会被创建（空文件），`CopyFileFromGuestToHost` 会成功复制一个空文件，最终返回空字符串。不会报错。

Q7: `runProgramInGuest` 和 `runScriptInGuest` 有什么区别?

| | runProgramInGuest | runScriptInGuest |

| --- | --- | --- |

| 执行方式 | 直接执行程序 | 通过解释器执行脚本文本 |

| 参数传递 | 支持单独传参 | 脚本作为整体传给解释器 |

| 典型用法 | 执行二进制程序 | 执行 shell 管道、复杂命令 |

| 输出捕获 | ❌ 不返回 | ❌ 不返回 |

在我们的修复中，两者都通过 `_run_with_output` 统一使用 `runScriptInGuest` + 文件重定向来捕获输出。

Q8: 虚拟机中的 VMware Tools 版本有要求吗?

没有严格要求。建议使用 VMware Workstation 自带的 VMware Tools 版本或 open-vm-tools（Linux 发行版内置）。较旧的版本可能缺少某些功能，但基本的 runProgramInGuest 和文件操作都支持。

Q9: 能不能同时操作多个虚拟机?

可以。vmrun 命令需要指定目标 VM 的 vmx 文件路径，对不同的 VM 分别调用即可。MCP Server 是无状态的，每次调用都独立执行。

Q10: 如果 MCP Server 断开了怎么办?

MCP Server 断开后，AI Agent 框架会自动重新启动它（配置为 `type: "local"` 时）。如果启动失败，需要检查：

- VMware Workstation 是否在运行

- vmrun.exe 路径是否正确

- 虚拟机是否处于开机状态

- VMware Tools 是否正常运行

---

10. 总结

10.1 最终方案

```
宿主机 AI Agent ↔ vmrun ↔ VMware Tools（VMCI 通道）↔ 内网虚拟机
                          ↑
                    零网络流量
                    不经过网卡
                    断网可用
```

10.2 踩坑记录

| 方案 | 结果 | 教训 |

| --- | --- | --- |

| vmrest Guest Operations API | ❌ Workstation Pro 不支持 | 即使 API 路径存在（返回 405），也不代表功能可用 |

| vmcli Guest run | ❌ 不返回 stdout | vmcli 的子命令参数映射有 bug |

| vmrun runProgramInGuest | ❌ 不返回 stdout | **不要轻信 AI 的能力描述，以实测为准** |

| **文件式 IPC（最终方案）** | ✅ 成功 | 简单可靠，路径短 |

10.3 一句话总结

> **用 VMware Tools 的 VMCI 通道替代 SSH，在宿主机上通过 vmrun 操作内网虚拟机，命令输出通过临时文件中转取回，实现 AI 对隔离虚拟机的完全控制，对内网零流量。**

附录

A. 参考文档

- VMware vmrun 命令参考

- MCP 协议规范

- OpenCode 配置文档

B. 知识库路径

所有操作经验已存储在 `[本地知识库]`：

C. 相关源码

关键文件：`D:\opencode\vmware-mcp\src\vmware_mcp\vmrun.py`

- `_run()` — 第 17 行，基础 vmrun 调用
- `_run_with_output()` — 第 129 行，文件式 IPC 捕获 stdout
- `run_program()` — 第 160 行，执行程序
- `run_script()` — 第 177 行，执行脚本

D.不在CS某N发布

- 若出现以上则默认为未授权搬运